

Documentación Sprint 0 | API Rest

Endpoints y Lógica de Negocio

En este documento se detalla el funcionamiento de la API REST desarrollada en **JavaScript con Express.js**, incluyendo ejemplos de datos, descripción de endpoints y explicación de la lógica de negocio contenida en el archivo "logica.js".

NOTA IMPORTANTE:

El objetivo de esta primera versión es definir la estructura base de la API REST y la lógica de negocio que gestiona usuarios, tipos de medición y mediciones.

Esta versión inicial se centra en la comunicación entre la aplicación Android y el servidor mediante peticiones **GET**, **POST** y **PUT**.

Autor: *Manuel Perez Garcia*

ÍNDICE

1.- Introducción.....	3
2.- Endpoints.....	4
2.1.- USUARIOS.....	4
2.1.1.- POST /users.....	4
2.1.2.- GET /users.....	4
2.1.3.- PUT /users/:id.....	5
2.2- MEDICIONES.....	6
2.2.1.- POST /mediciones.....	6
2.2.2.- GET /mediciones.....	6
2.3.- TIPO DE MEDICIÓN.....	7
2.3.1.- POST /tipomedicion.....	7
2.3.2.- PUT /tipomedicion/:id.....	7
2.3.3.- GET /tipomedicion.....	7
3.- Lógica de negocio: logica.js.....	8
Constructor lógico.....	8
Métodos principales.....	8
Usuarios.....	8
Mediciones.....	8
Tipos de medición.....	8
4.- Conexión a la base de datos: db.js.....	9
5.- Middleware de lectura: bodyManual.js.....	9
6.- Conclusión.....	9

1.- Introducción

Esta API permite **gestionar usuarios y mediciones de calidad del aire** en una base de datos MySQL.

Incluye los siguientes grupos de endpoints:

- `/users` → Gestión de usuarios
- `/mediciones` → Registro y consulta de mediciones
- `/tipomedicion` → Gestión de tipos de medición (por ejemplo: CO2, temperatura, humedad...)

El servidor está desarrollado con **Node.js + Express**, y la lógica de negocio separada en el módulo `logica.js` para mantener el código modular y reutilizable.

2.- Endpoints

2.1.- USUARIOS

2.1.1.- POST /users

Descripción: Crea un nuevo usuario en la base de datos.

Cuerpo JSON esperado:

```
{
  "nombre": "Juan",
  "apellidos": "Pérez",
  "telefono": "600123456",
  "gmail": "juan@example.com",
  "password": "1234"
}
```

Validaciones:

- Todos los campos son obligatorios.
- La contraseña se cifra con **bcrypt** antes de guardarse.

2.1.2.- GET /users

Descripción: Devuelve la lista completa de usuarios registrados.

Ejemplo de respuesta:

```
[
  {
    "id": 1,
    "nombre": "Juan",
    "apellidos": "Pérez",
    "telefono": "600123456",
    "gmail": "juan@example.com"
  },
  {
    "id": 2,
    "nombre": "Laura",
    "apellidos": "Martínez",
    "telefono": "612345678",
    "gmail": "laura@example.com"
  }
]
```

2.1.3.- PUT /users/:id

Descripción: Actualiza los datos de un usuario existente.

Parámetro de ruta: `:id` → ID del usuario a modificar.

Cuerpo JSON (ejemplo):

```
{  
  "telefono": "699999999",  
  "password": "nueva123"  
}
```

Validaciones:

- Al menos un campo debe enviarse.
- Si se incluye una nueva contraseña, se cifra automáticamente.

2.2- MEDICIONES

2.2.1.- POST /mediciones

Descripción: Registra una nueva medición asociada a un usuario y un tipo de medición.

Cuerpo JSON esperado:

```
{
  "tipomedicion": 1,
  "contador": 10,
  "medicion": 450,
  "user": 2,
  "hora": "2025-10-12 12:30:00",
  "localizacion": "Aula B203"
}
```

Validaciones:

- Campos obligatorios: `tipomedicion`, `medicion`, `user`, `hora`, `localizacion`.

2.2.2.- GET /mediciones

Descripción: Devuelve todas las mediciones con la información del usuario y del tipo de medición.

Respuesta JSON (ejemplo):

```
[
  {
    "id": 15,
    "medicion": 450,
    "contador": 10,
    "hora": "2025-10-12 12:30:00",
    "localizacion": "Aula B203",
    "user_id": 2,
    "user_nombre": "Laura",
    "user_apellidos": "Martínez",
    "tipo_id": 1,
    "tipo_medida": "CO2",
    "tipo_unidad": "ppm",
    "tipo_descripcion": "Nivel de dióxido de carbono en el aire"
  }
]
```

2.3.- TIPO DE MEDICIÓN

2.3.1.- POST /tipomedicion

Descripción: Crea un nuevo tipo de medición (ej. CO₂, temperatura...).

Cuerpo JSON esperado:

```
{
  "medida": "CO2",
  "unidad": "ppm",
  "txt": "Nivel de dióxido de carbono"
}
```

2.3.2.- PUT /tipomedicion/:id

Descripción: Actualiza un tipo de medición existente.

Cuerpo JSON (ejemplo):

```
{
  "txt": "Concentración de CO2 en partes por millón"
}
```

2.3.3.- GET /tipomedicion

Descripción: Devuelve todos los tipos de medición registrados.

Ejemplo de respuesta:

```
[
  {
    "id": 1,
    "medida": "CO2",
    "unidad": "ppm",
    "txt": "Nivel de dióxido de carbono"
  },
  {
    "id": 2,
    "medida": "Temperatura",
    "unidad": "°C",
    "txt": "Temperatura ambiental"
  }
]
```

3.- Lógica de negocio: logica.js

La clase logica.js contiene todas las funciones que interactúan con la base de datos MySQL.

Estas funciones son **puras y reutilizables**, ya que reciben la conexión `db` como parámetro.

Constructor lógico

La conexión no se define en esta clase; se inyecta desde `db.js` para mantener la independencia del código.

Métodos principales

Usuarios

- `createUser(db, datos)` → Inserta un nuevo usuario cifrando la contraseña.
- `listUsers(db)` → Devuelve todos los usuarios (sin contraseñas).
- `updateUser(db, id, campos)` → Actualiza uno o varios campos del usuario.

Mediciones

- `createMedicion(db, datos)` → Inserta una nueva medición.
- `listMediciones(db)` → Devuelve todas las mediciones con JOIN sobre usuarios y tipos de medición.

Tipos de medición

- `createTipoMedicion(db, datos)` → Inserta un nuevo tipo de medición.
- `updateTipoMedicion(db, id, campos)` → Actualiza campos de un tipo de medición existente.
- `listTipoMedicion(db)` → Lista todos los tipos registrados.

4.- Conexión a la base de datos: db.js

Se usa el paquete **mysql2** para conectar la API con la base de datos **medicionesaire**:

```
const mysql = require('mysql2');

const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'medicionesaire',
  port: 3306
});
```

La conexión se establece al iniciar el servidor y se reutiliza en toda la aplicación.

5.- Middleware de lectura: bodyManual.js

Este middleware permite **parsear manualmente el cuerpo del body** en peticiones **POST** y **PUT**, asegurando compatibilidad con clientes que no envían **Content-Type: application/json**.

Además, se utiliza como **medida de seguridad para no perder información recibida**, ya que en caso de que el JSON no se parsee correctamente o llegue con formato incorrecto, el middleware conserva los datos crudos y evita que se pierdan durante el proceso, permitiendo una respuesta controlada sin afectar al servidor.

6.- Conclusión

Esta versión base de la API REST permite:

- Registrar y consultar usuarios, tipos de medición y mediciones.
- Mantener la lógica separada y modular.
- Conectarse fácilmente desde una aplicación Android mediante peticiones HTTP estándar.

Esta estructura facilita la ampliación futura del proyecto, añadiendo autenticación, filtros o dashboards de análisis.